

Oblig 1

Oppgave 1.1 - Forklar hva JRE (Java Runtime Environment) og JDK (Java Development Kit) er.

- JRE er et system som er bygd opp av programvare komponenter som brukes til å kompilere og kjøre Java programmer. JRE er hovedsakelig bygd opp av JVM(Java Virtual Machine) som er programvaren som oversetter Java koden til lesbar kode for datamaskiner, og Java Library som er et sett med predefinert Java kode og funksjoner.
- JDK er et komplett utviklersystem for Java som blant annet inneholder JRE. I tillegg til dette inneholder også JDK verktøy for utvikling, feilsøking og monitorering av Java applikasjoner.

Oppgave 1.2 - Forklar flyten av hvordan java-applikasjoner blir bygget og kjørt. Knytt også forklaringen din opp mot terminal-kommandoene, javac og java.

Først starter man med å skrive en Javakode. Dette kan i teorien gjøres i et vilkårlig tekstprogram slik som 'notepad', men i dag har vi mer sofistikerte IDE systemer med integrerte funksjoner, som gjør det vesentlig mye mer enklere og effektivt å skrive/teste kode. Et eksempel på dette er IntelliJ IDEA.

Når du har skrevet ferdig en Javakode, så vil dette bli lagret som en Java-fil (din_klasse.java). For at denne koden til slutt skal kunne bli lesbar for en maskin (binær), så må den gjennom noen steg. Det første er at vi må kompilere koden ned til *bytecode*. For å kompilere Javakode må vi bruke javac (Java compiler) kommandoen i en terminal og definerer hvilken Javakode vi ønsker å kompilere. Når vi kjører dette på vår Javakode blir det generert en ny tilhørende klasse fil som er bygget opp av *bytecode*. Denne filen er plattform uavhengig og kan dermed kjøres på alle maskiner som kjører Java. Nå er koden klar for å bli kjørt, og for å kjøre Javakoden vår brukes JVM(Java Virtual Machine). Denne programvaren leser *bytecode* fra den kompilerte filen og oversetter den til binær data som maskinen klarer å forstå. JVM starter å oversette til maskinen når du bruker Java kommandoen (java mainclass) i en terminal til å kjøre programmet.

Oppgave 1.3 - Forklar forskjellen på Compile-time og Run-time errors. Gi noen eksempler.

Compile-time error vedrører feil som oppstår under kompilering av koden. Med dette menes typiske menneskelige feil, slik som skrivefeil, feil i syntax og andre tilsvarende avvik fra kodespråkets struktureringskrav. Dette er feil som blir oppdaget under kompilering av kode, og alle slike feil må løses før noe som helst blir videre kompilert (Om det er feil på en linje blir heller ingenting annet kompilert).

Typiske eksempler på dette er en glemt semikolon, udeklarte variabler og konkrete skrivefeil.

Run-time errors er feil som oppstår når programmet kjører. Dette vil si at kompileringen er blitt fullført uten feil, kildekoden har blitt kompilert til *bytecode* og JVM har startet programmet. Her er det da feil som vanligvis får programmet til å krasje eller utfører uforventede handlinger. Typiske tilfeller som fører til dette er blant annet steder hvor programmet prøver å dele noe på 0, om programmet prøver å kalle på noe utenfor lengden på en liste eller om programmet prøver å hente ut informasjon fra en referanse med 'null' verdi.

Dette er et eksempel på Javakode hvor en string variabel har blitt definert med 'null' verdi og etterpå så prøver programmet å hente ut lengden på stringen til denne variabelen.

```
public class NullReferenceExample {
    public static void main(String[] args) {
        String str = null;

        int length = str.length(); // This will result in a NullPointerException
    }
}
```

Oppgave 1.4 - Forklar forskjellen på en klasse og et objekt. Vis gjerne et lite eksempel på hvordan man definerer en klasse, og oppretter et objekt.

Java er det vi kaller et objekt orientert program (OOP). Med dette så menes at fundamentalt sett så er Javakode bygd opp av klasser og objekter. Disse to henger tett sammen og du kan ikke ha den ene uten den andre, men det kreves også en strukturert fremgang for at dette skal gi mening i programmering. Vi bruker nemlig klasser for å lage objekter.

Klasser: Klasser kan ansees som plantegningen for å lage objekter. Det er her vi definerer hva objektene skal representere, hvilke verdier de skal kunne holde på og hvilke handlinger de skal kunne utføre. Instanser er objekter som blir laget basert på innhold og oppsettet i en klasse.

I en klasse definerer du hvilke attributter et objekt kan få. Om vi skal ta for oss en klasse som representerer biler så vil typiske attributter være ting som modell, farge, produksjonsår, osv.

Metoder er definisjoner av hvilke handlinger et objekt kan utføre. I en klasse som omhandler biler vil typiske eksempler på dette være egenskapen til å starte, stoppe, akselerere, bremse, osv.

Dette er et eksempel på hvordan en klasse som omhandler biler kan skrives med Javakode:

```
public class Car {
    // Her defineres attributter for bilen
    String model;
    String color;

    // Her definerer vi hvilke handlinger bil skal kunne utføre (metoder)
    void start() {
        System.out.println("The car is starting.");
    }

    void stop() {
        System.out.println("The car is stopping.");
    }
}
```

Objekt: Et objekt er instanser som blir opprettet basert på innholdet til en klasse. Det er her vi spesifiserer hvilke verdier de forskjellige attributtene skal holde for hvert objekt og hvilke handlinger det gitte objektet skal utføre.

I eksempelet med biler vil da et objekt få spesifisert sine attributt verdier. Slik som at model attributtet blir satt til bilmerke, farge attributtet blir satt til bilens farge, osv.

Klassens metoder blir så lagt til i objektet slik at objektet utfører de ønskede handlingene.

Her er et eksempel hvor det blir opprettet to objekter basert på predefinert kode i den overordnede klassen:

```
public class Main {
    public static void main(String[] args) {
        // Her oppretter vi objektene ved å gi de et variabel navn.
        Car myCar = new Car();
        Car anotherCar = new Car();

        // Her definerer vi objektets attributter.
        myCar.model = "Toyota";
        myCar.color = "Blue";

        anotherCar.model = "Honda";
        anotherCar.color = "Red";

        // Her definerer vi de forskjellige handlingene som vi ønsker at
        // objektet skal utføre.
        myCar.start();
        anotherCar.stop();
    }
}
```

Litteraturliste:

Schildt, H. (2022). Java: A Beginner's Guide (9. Utg). McGraw Hill. ISBN: 978-1260463552

OpenAI. (2024). ChatGPT(ChatGPT 3.5)

<https://chat.openai.com/share/b93f6007-9a10-467e-b52b-066683a62c09>